



9.2. Expression Evaluation using Stacks

Previous Sections:

 [9. Stacks](#)

 [9.1. Operations on Stacks](#)

Contents:

[1. Forms of Arithmetic Expressions](#)

[2. Evaluating Postfix Expressions](#)

[3. Evaluating Prefix Expressions](#)

[Summary](#)

[References](#)

Here, we will reuse the Stack object implemented in the previous section:

[Stack.py](#)

One of the most important applications of stacks is **expression evaluation**.

Whenever we write a mathematical expression, such as: `1 + 2 * 3`, humans can easily determine which operation should be performed first because we understand operator precedence.

A computer, however, must explicitly follow a sequence of rules. This is where stacks become extremely useful. The general idea is simple:

1. Store parts of the expression in a stack.
2. Remove elements when an operation needs to be performed.
3. Calculate intermediate results.
4. Push the results back into the stack.
5. Continue until a final answer remains.

In most programming languages, expressions are represented as strings and consist of:

- **Operands:** Values that participate in calculations. Examples are `1; 2; 3; a; b; x`
- **Operators:** Symbols that perform calculations. Examples are `++; --; *; /`
 - Most arithmetic operators are **binary operators**, meaning they require two operands.
 - For example, `2 + 3` requires: `Operand 1 = 2; Operand 2 = 3; Operator = +`
- **Parentheses:** Parentheses determine evaluation priority.
 - Consider the following expressions: `(1 + 2) * 3` → Result: `9` because `(1 + 2) = 3 ⇒ 3 * 3 = 9`.
 - Now compare it with: `1 + 2 * 3` → Result: `7`, because multiplication has higher precedence: `2 * 3 = 6 ⇒ 1 + 6 = 7`

1. Forms of Arithmetic Expressions

An arithmetic expression can be written in three different notations.

- **Infix Notation:** The operator is placed between the operands: `a + b`; `1 + 2`; `A * B`. This is the notation humans commonly use.
- **Prefix Notation:** The operator appears before the operands: `+ a b`; `+ 1 2`; `* A B`
- **Postfix Notation:** The operator appears after the operands: `a b +`; `1 2 +`; `A B *`

Example - Given the following expression: `(1 + 2) * 3`

- Infix: `(1 + 2) * 3`

- Prefix: `* + 1 2 3`
- Postfix: `1 2 + 3 *`

Another Example - Expression: `1 + 2 * 3`

- Infix: `1 + 2 * 3`
- Prefix: `+ 1 * 2 3`
- Postfix: `1 2 3 * +`

Why Convert Expressions? Computers find infix expressions more difficult to evaluate because they must constantly check: Parentheses; Operator precedence; Associativity

Prefix and postfix expressions remove this ambiguity entirely. For this reason, infix expressions are often converted into prefix or postfix before evaluation.

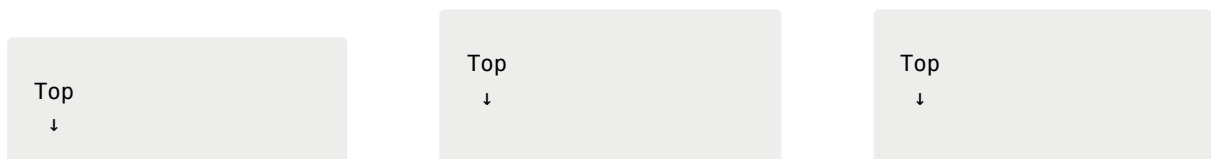
2. Evaluating Postfix Expressions

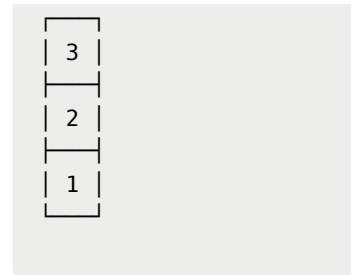
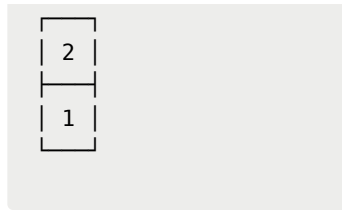
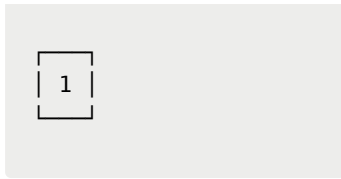
Postfix expressions are evaluated from **left to right**. We use a stack to temporarily store operands. The process follows these rules:

- **Rule 1:** If an operand is encountered → **Push it onto the stack**
- **Rule 2:** If an operator is encountered → **Pop the required operands. Perform the calculation. Push the result back.**
- Repeat until the expression ends. The final value remaining in the stack is the answer.

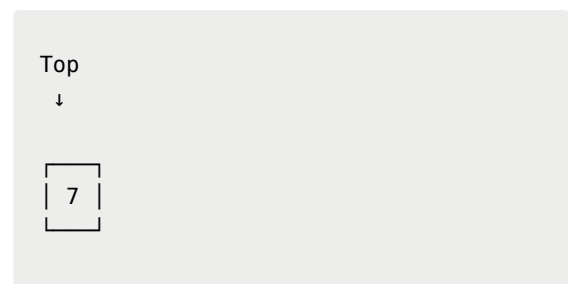
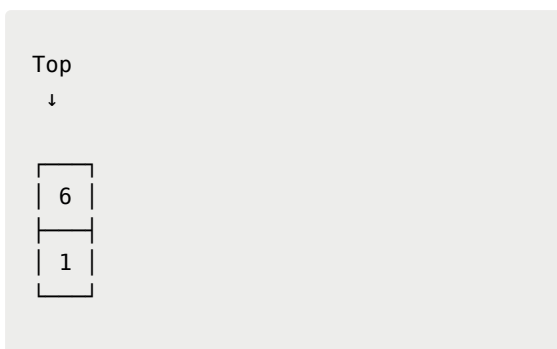
Example: Evaluating `1 + 2 * 3`

- First convert the infix expression `1 + 2 * 3` to postfix: `1 2 3 * +`
- **Step 1** → Push 1 onto stack.
- Step 2 → Push 2 onto stack.
- Step 3 → Push 3 onto stack.





- Step 4: Encounter an operator `*` → Pop two operands: 3 and 2. Calculate: `2 * 3 = 6`. Push result.
- **Step 5:** Encounter an operator `+` → Pop two operands: 6 and 1. Calculate: `1 + 6 = 7`. Push result.



- **Final Result:** Only one element remains is `7`. Therefore: `1 + 2 * 3 = 7`

To perform calculations, we can create a function that evaluates two operands and an operator.

```
def evaluate(number1, number2, operator):  
    if operator == "+":  
        return number1 + number2  
    elif operator == "-":  
        return number1 - number2  
    elif operator == "*":  
        return number1 * number2  
    else:  
        return number1 / number2
```

- Postfix Evaluation Algorithm

```
def evaluate_postfix(postfix_list):
    stack = Stack()

    operators = ["+", "-", "*", "/"]

    for entry in postfix_list:
        if entry in operators:
            b = stack.pop()
            a = stack.pop()

            temp = evaluate(a, b, entry)
            stack.push(temp)

        else:
            stack.push(int(entry))

    return stack.get_top()
```

```
postfix = ["1", "2", "3", "*", "+"]

print(evaluate_postfix(postfix))
```

7

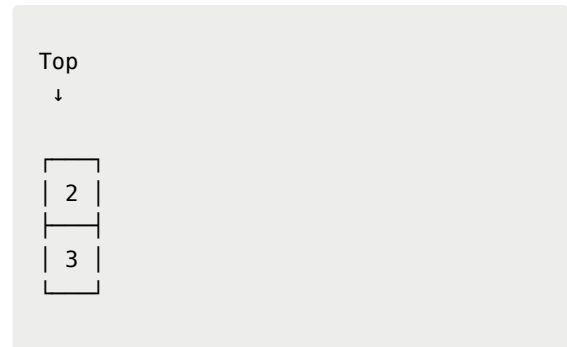
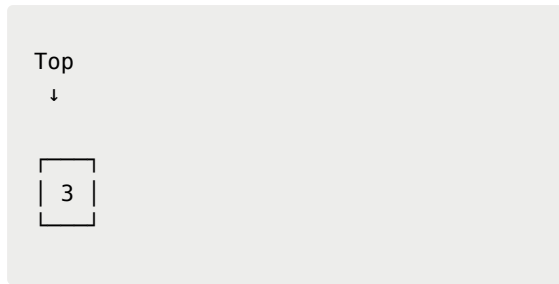
3. Evaluating Prefix Expressions

Prefix expressions are evaluated in a very similar manner. The only difference is: ***Read the expression from RIGHT to LEFT***. Everything else remains the same.

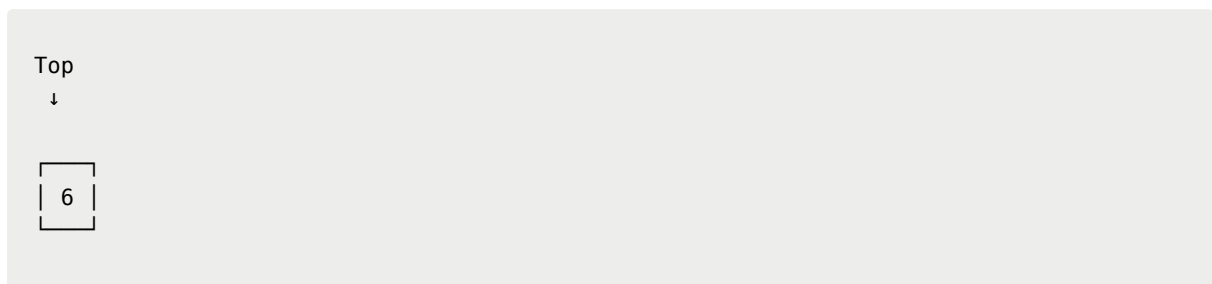
- Operands are pushed onto the stack.
- Operators cause operands to be popped.
- Results are pushed back.

Example: Evaluating `+ 1 * 2 3` . We read it from right to left.

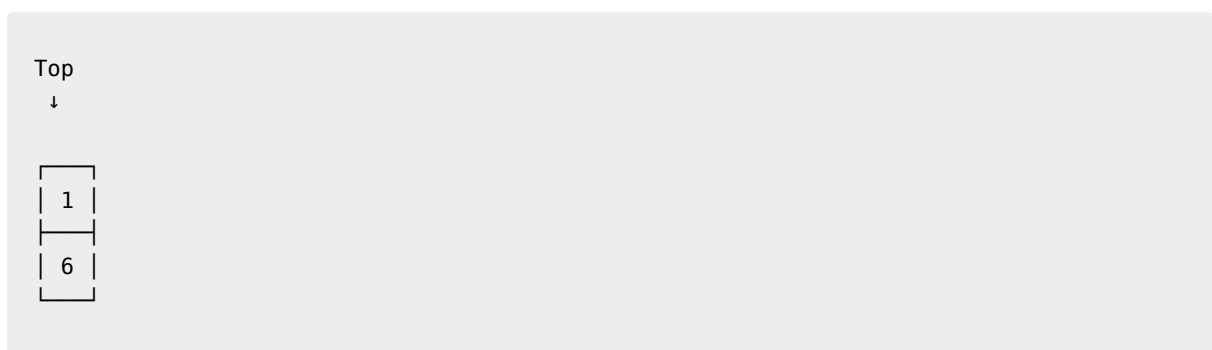
- Step 1: Push 3
- Step 2: Push 2



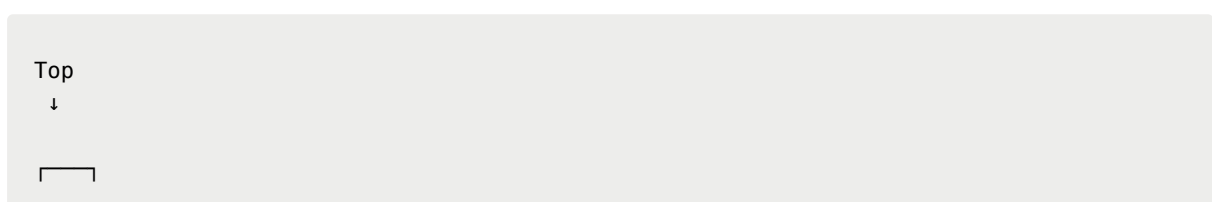
- Step 3: Encounter $*$. Pop operands 2 and 3 → Calculate: $2 * 3 = 6$. Push result.



- Step 4: Push 1



- Step 5: Encounter $+$. Pop operands 1 and 6 → Calculate: $1 + 6 = 7$. Push result.



7

- Final Result is 7 → Thus: $+ 1 * 2 3 = 7$

Prefix Evaluation Algorithm

```
def evaluate_prefix(prefix_list):  
    stack = Stack()  
    operators = ["+", "-", "*", "/"]  
  
    for entry in reversed(prefix_list):  
  
        if entry in operators:  
            a = stack.pop()  
            b = stack.pop()  
  
            temp = evaluate(a, b, entry)  
            stack.push(temp)  
  
        else:  
            stack.push(int(entry))  
  
    return stack.get_top()
```

```
prefix = ["+", "1", "*", "2", "3"]  
  
print(evaluate_prefix(prefix))
```

7

Summary

Now that we know how postfix and prefix expressions can be evaluated efficiently using stacks, the next step is learning how to **convert infix expressions into postfix and prefix expressions**, which is where stacks become even more powerful.

(I have decided to put it to another dedicated section)

References

https://drive.google.com/file/d/1yBJEh1C9V-nXZ7TF80JtWvozfaDYRq5B/view?usp=drive_link

<https://www.geeksforgeeks.org/dsa/expression-evaluation/>

Next Section:

 [9.3. Converting Infix Expressions using Stacks](#)